

Nhập/xuất, chú thích, trích dẫn & Number, String

Mục tiêu sau buổi thực hành

- Sử dụng thành thạo `input()`, `print()` (kèm `sep/end`) và các cách định dạng chuỗi: f-string, `str.format()`, toán tử `%`.
- Phân biệt các kiểu chú thích, các kiểu trích dẫn chuỗi và ký tự escape thông dụng.
- Làm chủ kiểu dữ liệu số (`int`, `float`, `complex`) cùng các hàm built-in `abs()`, `round()`, `pow()`, `divmod()`.
- Vận dụng thành thạo String: tính bất biến, indexing/slicing, các phương thức `split()`, `join()`, `strip()`, `replace()`, `upper()/lower()`, `find()`, `count()`, `startswith()/endswith()`.

Lưu ý về phạm vi kiến thức: Buổi 2 chỉ sử dụng biến, toán tử, định danh (đã học Buổi 1) và nội dung Number/String của chính buổi này. Các bài tập chưa dùng `if/elif/else`, chưa dùng vòng lặp `for/while` và chưa tự định nghĩa hàm. Những phần việc cần rẽ nhánh (ví dụ xét đủ 3 trường hợp nghiệm phương trình bậc hai, tính tiền điện theo bậc thang) sẽ được hoàn thiện lại sau - ở đây sinh viên chỉ xử lý các trường hợp có thể giải quyết bằng biểu thức toán tử/so sánh thuần túy.

Hoạt động 1: Nhập/xuất dữ liệu & định dạng chuỗi

Bài tập 1.1 - `input()` và ép kiểu:

```
ho_ten = input("Nhập họ tên: ")
nam_sinh = int(input("Nhập nam sinh: "))
diem_tb = float(input("Nhập điểm trung bình: "))
```

Yêu cầu: giải thích vì sao phải ép kiểu `int()/float()` cho `nam_sinh` và `diem_tb`, trong khi `ho_ten` thì không cần.

Bài tập 1.2 - `print()` với `sep/end`:

```
print("Python", "la", "ngon", "ngu", "lap trinh", sep="-")
print("Dong 1", end=" | ")
print("Dong 2")
```

Yêu cầu: thử đổi `sep` thành nhiều ký tự khác nhau (`"`, `" "`, `"\n"`) và quan sát kết quả rồi giải thích.

Bài tập 1.3 - So sánh 3 cách định dạng chuỗi:

Với 3 biến đã nhập ở Bài tập 1.1, in ra một dòng thông tin bằng **cả 3 cách**:

```
# f-string
print(f"Ho ten: {ho_ten} - Nam sinh: {nam_sinh} - DTB: {diem_tb:.2f}")

# str.format()
print("Ho ten: {} - Nam sinh: {} - DTB: {:.2f}".format(ho_ten, nam_sinh, diem_tb))

# toán tử %
```

```
print("Ho ten: %s - Nam sinh: %d - DTB: %.2f" % (ho_ten, nam_sinh, diem_tb))
```

Thảo luận: 3 cách trên cho kết quả giống nhau, vậy vì sao Python hiện nay khuyến khích dùng f-string hơn?

Hoạt động 2: Chú thích & các kiểu trích dẫn

Bài tập 2.1 - Chú thích:

Thêm đầy đủ 3 loại chú thích vào một đoạn code có sẵn:

```
# Chu thích mot dong: khai bao thong tin sinh vien
"""
Chu thích/docstring nhieu dong:
Chuong trinh quan ly diem sinh vien - Buoai 2
"""
ho_ten = "Tran Thi B" # bien luu ho ten
```

Bài tập 2.2 - Các kiểu trích dẫn & escape:

Dự đoán rồi chạy thử để kiểm tra kết quả in ra của từng dòng:

```
s1 = 'Xin chao'
s2 = "Ban co khoe khong?"
s3 = '''Day la
mot chuoai
nhieu dong'''
s4 = "Duong dan: C:\\Python\\data"
s5 = r"Duong dan raw: C:\Python\data"
s6 = "Toi ten la \"Nam\", con ban ten gi?"

print(s1); print(s2); print(s3); print(s4); print(s5); print(s6)
```

Yêu cầu: giải thích khác biệt giữa s4 và s5 (raw string dùng khi nào?).

Hoạt động 3: Number - int, float, complex & hàm built-in

Bài tập 3.1 - Các kiểu số & chuyển đổi:

```
so_nguyen = 15
so_thuc = 4.2
so_phuc = 3 + 4j

print(type(so_nguyen), type(so_thuc), type(so_phuc))
print(float(so_nguyen)) # ep int -> float
print(int(so_thuc))     # ep float -> int (cat phan thap phan)
```

Bài tập 3.2 – Hàm built-in xử lý số:

```
a = -7
b = 2.6789
c, d = 17, 5
```

```
print(abs(a))          # gia tri tuyet doi
print(round(b))        # lam tron
print(round(b, 2))     # lam tron 2 chu so thap phan
print(pow(c, 2))       # c mu 2
print(divmod(c, d))    # tra ve (thuong, du) dang tuple
```

Yêu cầu: so sánh `pow(c, 2)` với `c ** 2` (toán tử đã học ở Buổi 1) - hai cách này có luôn cho kết quả giống nhau không? Tại sao?

Bài tập 3.3 - Vận dụng: Tính nghiệm phương trình bậc hai (trường hợp có 2 nghiệm phân biệt):

Cho phương trình $a \cdot x^2 + b \cdot x + c = 0$ với bộ số đã chọn trước sao cho delta luôn dương (ví dụ $a=1, b=-3, c=2$):

```
import math

a, b, c = 1, -3, 2
delta = b ** 2 - 4 * a * c
x1 = (-b + math.sqrt(delta)) / (2 * a)
x2 = (-b - math.sqrt(delta)) / (2 * a)

print(f"Delta = {delta}")
print(f"Nghiem x1 = {round(x1, 2)}, x2 = {round(x2, 2)}")
```

Ghi chú: Bài tập này mới chỉ xử lý trường hợp delta dương. Việc xét đầy đủ $\text{delta} > 0 / = 0 / < 0$ cần dùng `if/elif/else` nên sẽ được hoàn thiện sau.

Hoạt động 4: String - indexing, slicing & phương thức xử lý

Bài tập 4.1 - Indexing & slicing:

```
cau = "Lap trinh Python rat thu vi"

print(cau[0])          # ky tu dau tien
print(cau[-1])         # ky tu cuoi cung
print(cau[4:10])       # cat tu vi tri 4 den truoc vi tri 10
print(cau[:8])         # tu dau den vi tri 8
print(cau[11:])        # tu vi tri 11 den het
print(cau[::-1])       # dao nguoc chuoai
```

Yêu cầu: dùng `cau[::-1]` để in ra chuỗi đảo ngược, sau đó kiểm tra xem `cau` có phải là palindrome không bằng biểu thức so sánh: `cau == cau[::-1]`.

Bài tập 4.2 - Tính bất biến (immutable):

```
ten = "Nam"
# Thu gan lai mot ky tu: ten[0] = "T" -> quan sat loi TypeError
ten_moi = "T" + ten[1:]
print(ten_moi)
```

Bài tập 4.3 - Các phương thức xử lý chuỗi thường dùng:

```
cau = "  Toi dang HOC Python rat vui  "

print(cau.strip())           # bo khoang trang 2 dau
print(cau.strip().upper())   # in hoa toan bo
print(cau.strip().lower())   # in thuong toan bo
print(cau.strip().replace("HOC", "hoc"))
print(cau.strip().split())    # tach thanh danh sach cac tu
print(len(cau.strip().split())) # dem so tu trong cau
print(cau.count("o"))         # dem so lan xuat hien ky tu 'o'
print(cau.find("Python"))     # vi tri bat dau cua "Python"
print(cau.strip().startswith("Toi"))
print(cau.strip().endswith("vui"))
print("-".join(["Python", "that", "thu", "vi"]))
```

Bài tập 4.4 - Vận dụng: Chuẩn hóa họ tên:

Cho chuỗi họ tên nhập vào có khoảng trắng thừa và chữ hoa/thường lộn xộn, ví dụ " nguyên văn an ". Yêu cầu dùng `strip()`, `split()`, `join()` để: xóa khoảng trắng thừa ở đầu/cuối và giữa các từ, sau đó viết hoa chữ cái đầu mỗi từ bằng phương thức `title()`.

```
ho_ten_tho = "  nguyên văn an "
ho_ten_sach = " ".join(ho_ten_tho.split()).title()
print(ho_ten_sach)  # Nguyen Van An
```

Hoạt động 5: Mini project - Đăng ký thông tin cá nhân

Yêu cầu: Viết chương trình cho phép nhập các thông tin sau và kiểm tra dữ liệu cơ bản, **chỉ dùng biến, toán tử và các phương thức String/Number đã học** (chưa dùng `if`, kết quả kiểm tra chỉ cần in ra biến `True/False`):

- Họ tên → chuẩn hóa theo Bài tập 4.4.
- Số điện thoại (dạng chuỗi) → kiểm tra đủ 10 ký tự bằng `len()`.
- Email → kiểm tra có chứa ký tự "@" bằng toán tử `in` (đã học Buổi 1).

```
ho_ten = input("Nhập họ tên: ")
sdt = input("Nhập số điện thoại: ")
email = input("Nhập email: ")

ho_ten_chuan = " ".join(ho_ten.split()).title()
sdt_hop_le = len(sdt) == 10
email_hop_le = "@" in email

print(f"Họ tên (đã chuẩn hóa): {ho_ten_chuan}")
print(f"Số điện thoại hợp lệ (đủ 10 ký tự)? {sdt_hop_le}")
print(f>Email hợp lệ (có ký tự @)? {email_hop_le}")
```

Ghi chú: Chương trình mới chỉ báo cáo kết quả kiểm tra (đúng/sai), chưa thể từ chối/yêu cầu nhập lại khi dữ liệu sai - phần đó cần `if` kết hợp vòng lặp `while`, sẽ hoàn thiện sau.

Sản phẩm nộp cuối buổi

1. File hoat_dong_1_2.py (nhập/xuất, định dạng chuỗi, chú thích, trích dẫn).
2. File hoat_dong_3_4.py (Number, String).
3. File dang_ky_thong_tin.py (mini project Hoạt động 5).

- Chụp ảnh thực thi thành công các hoạt động
- Trả lời các nội dung đã yêu cầu.
- Link Github
- Đặt trong file PDF để nộp bài